Nama: Muhammad Rafly Romeo Nasution
Kelas: 3KA25
NPM: 10123875

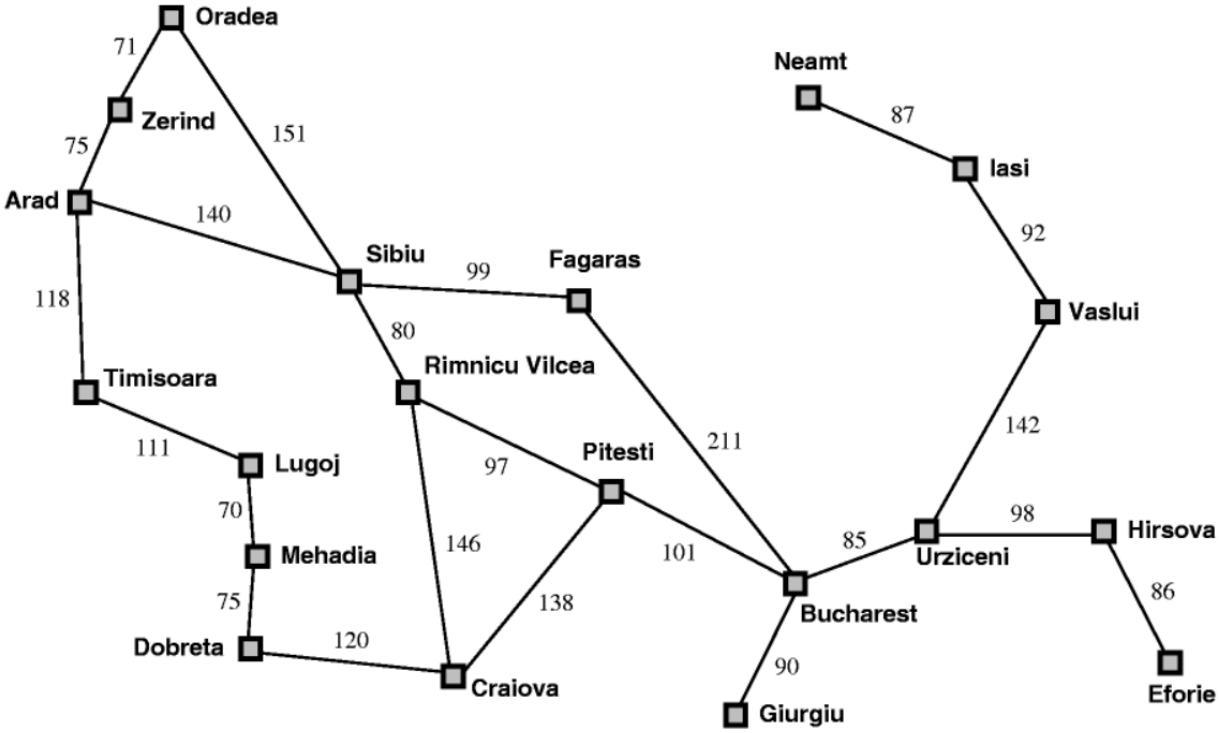
MK Praktikum Unggulan Universitas Gunadarma

Mata Kuliah: Praktikum Terapan Teori Graf (Tingkat 3)

Pertemuan IV

Studi Kasus

Dalam studi kasus kali ini, akan melakukan pencarian jarak terdekat dari kota Arad ke Bucharest.




```
# Define graph with tipe data dictionary
graph = {
  'A' : ['Z','S', 'T'],
  'Z' : ['O'],
  'O' : ['S1'],
  'S1' : ['F','R'],
  'F' : ['B'],
  'B' : [],
  'R' : [],
  'S' : ['O1', 'F1', 'R1'],
  'O1' : [],
```

```
  'F1' : ['B1'],
  'B1' : [],
  'R1' : ['C', 'P'],
  'C' : [],
  'P' : [],
  'T' : ['L'],
  'L' : ['M'],
  'M' : ['D'],
  'D' : []
}
```

```
visited = []   # List for visited nodes
queue = []     # Initialize a queue
```

```
def bfs(visited, graph, node, target):
  visited.append(node)
  queue.append(node)

  while queue:
    m = queue.pop(0)
    print (m, end = " ")

    if m == target:
      print(f"\nTarget {target} ditemukan!")
      break

    for neighbour in graph[m]:
      if neighbour not in visited:
        visited.append(neighbour)
        queue.append(neighbour)
```

```
print("Following is the Breadth-First Search for target 'B1'")
bfs(visited, graph, 'A', 'B1')
```

```
Following is the Breadth-First Search for target 'B1'
A Z S T O O1 F1 R1 L S1 B1
Target B1 ditemukan!
```

> **Logika Pemrograman BFS Searching:**
> Algoritma Breadth-First Search (BFS) bekerja dengan cara mengeksplorasi graf secara melebar. Node awal dimasukkan ke dalam antrean (`queue`) dan daftar kunjungan (`visited`). Selama antrean tidak kosong, program akan mengeluarkan elemen pertama dari antrean, mencetaknya, dan mengecek apakah *node* tersebut adalah `target` yang dicari. Jika ya, pencarian dihentikan menggunakan instruksi `break`. Jika tidak, program akan mengecek semua tetangganya; tetangga yang belum pernah dikunjungi akan dimasukkan ke dalam daftar kunjungan dan ditambahkan ke antrean untuk dievaluasi pada giliran selanjutnya.

Inisialisasi DFS

```
visited_dfs = []   # List for visited nodes
```

Fungsi DFS dan Pemanggilan DFS

```
def dfs(visited, graph, node, target):
  if node not in visited:
    print (node, end = " ")
    visited.append(node)

    if node == target:
      print(f"\nTarget {target} ditemukan!")
      return True

    for neighbour in graph[node]:
      if dfs(visited, graph, neighbour, target):
        return True

  return False

print("Following is the Depth-First Search for target 'B1'")
dfs(visited_dfs, graph, 'A', 'B1')
```

```
Following is the Depth-First Search for target 'B1'
A Z O S1 F B R S O1 F1 B1
Target B1 ditemukan!
True
```

> **Logika Pemrograman DFS Searching:**
> Berbeda dengan BFS, Depth-First Search (DFS) mengeksplorasi graf secara mendalam menggunakan rekursi. Fungsi `dfs` akan mencetak *node* saat ini dan memasukkannya ke daftar kunjungan (`visited`). Program kemudian mengecek apakah *node* tersebut adalah `target`. Jika target ditemukan, program mengembalikan nilai `True` yang secara berantai akan menghentikan seluruh proses rekursi. Jika bukan target, program akan menelusuri secara mendalam ke tetangga pertama yang belum dikunjungi hingga mencapai ujung *branch*, sebelum mundur (*backtrack*) untuk mengeksplorasi cabang lainnya.

Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/